

Weekly Assignment 2: Controllers & WebSocket Configurations (Real-Time Chat Application)

This document contains the source code for the **Controllers** and **WebSocket Configurations** layer of the Real-Time Chat Application, which manage REST requests, security configurations, and live STOMP messaging channels.

1. REST Controllers

AuthController.java

Handles user authentication: signup, login, and signout, updating online/offline presence status.

```
package com.chatapp.controller;

import com.chatapp.dto.JwtResponse;
import com.chatapp.dto.LoginRequest;
import com.chatapp.dto.SignupRequest;
import com.chatapp.model.User;
import com.chatapp.model.UserStatus;
import com.chatapp.repository.UserRepository;
import com.chatapp.security.jwt.JwtUtils;
import com.chatapp.security.services.UserDetailsImpl;
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;
import java.util.HashMap;
import java.util.Map;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired
    AuthenticationManager authenticationManager;

    @Autowired
    UserRepository userRepository;
```

```

    @Autowired
    PasswordEncoder encoder;

    @Autowired
    JwtUtils jwtUtils;

    @PostMapping("/login")
    public ResponseEntity<?> authenticateUser(@Valid @RequestBody LoginRequest loginRequest) {
        Authentication authentication = authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(loginRequest.getUsername(), loginRequest.getPassword())
        );

        SecurityContextHolder.getContext().setAuthentication(authentication);
        String jwt = jwtUtils.generateJwtToken(authentication);

        UserDetailsImpl userDetails = (UserDetailsImpl) authentication.getPrincipal();

        userRepository.findByUsername(userDetails.getUsername()).ifPresent(user -> {
            user.setStatus(UserStatus.ONLINE);
            userRepository.save(user);
        });

        return ResponseEntity.ok(new JwtResponse(
            jwt,
            userDetails.getId(),
            userDetails.getUsername(),
            userDetails.getEmail(),
            userDetails.getAvatarUrl()
        ));
    }

    @PostMapping("/register")
    public ResponseEntity<?> registerUser(@Valid @RequestBody SignupRequest signUpRequest) {
        if (userRepository.existsByUsername(signUpRequest.getUsername())) {
            Map<String, String> err = new HashMap<>();
            err.put("message", "Error: Username is already taken!");
            return ResponseEntity.badRequest().body(err);
        }

        if (userRepository.existsByEmail(signUpRequest.getEmail())) {
            Map<String, String> err = new HashMap<>();
            err.put("message", "Error: Email is already in use!");
            return ResponseEntity.badRequest().body(err);
        }

        User user = new User();
        user.setUsername(signUpRequest.getUsername());
        user.setEmail(signUpRequest.getEmail());
        user.setPassword(encoder.encode(signUpRequest.getPassword()));
        user.setStatus(UserStatus.ONLINE);
        user.setAvatarUrl(signUpRequest.getAvatarUrl());
    }

```

```

    userRepository.save(user);
}

Map<String, String> msg = new HashMap<>();
msg.put("message", "User registered successfully!");
return ResponseEntity.ok(msg);
}

@PostMapping("/logout")
public ResponseEntity<?> logoutUser() {
    Authentication auth = SecurityContextHolder.getContext().getAuthentication();
    if (auth != null && auth.getPrincipal() instanceof UserDetailsImpl) {
        UserDetailsImpl userDetails = (UserDetailsImpl) auth.getPrincipal();
        userRepository.findByUsername(userDetails.getUsername()).ifPresent(user -> {
            user.setStatus(UserStatus.OFFLINE);
            userRepository.save(user);
        });
    }
    SecurityContextHolder.clearContext();
    Map<String, String> msg = new HashMap<>();
    msg.put("message", "Logged out successfully!");
    return ResponseEntity.ok(msg);
}
}

```

ChatRoomController.java

Exposes REST endpoints to query public chat rooms or create a new public room.

```

package com.chatapp.controller;

import com.chatapp.model.ChatRoom;
import com.chatapp.repository.ChatRoomRepository;
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/rooms")
public class ChatRoomController {

    @Autowired
    private ChatRoomRepository chatRoomRepository;

    @GetMapping

```

```

    public ResponseEntity<List<ChatRoom>> getAllRooms() {
        List<ChatRoom> rooms = chatRoomRepository.findAllByNameAsc();
        return ResponseEntity.ok(rooms);
    }

    @PostMapping
    public ResponseEntity<?> createRoom(@Valid @RequestBody ChatRoom chatRoom) {
        if (chatRoomRepository.findByName(chatRoom.getName()).isPresent()) {
            Map<String, String> err = new HashMap<>();
            err.put("message", "Error: Room name already exists!");
            return ResponseEntity.badRequest().body(err);
        }

        chatRoom.setPrivate(false);
        ChatRoom savedRoom = chatRoomRepository.save(chatRoom);
        return ResponseEntity.ok(savedRoom);
    }
}

```

UserController.java

Exposes user catalog details.

```

package com.chatapp.controller;

import com.chatapp.model.User;
import com.chatapp.repository.UserRepository;
import com.chatapp.security.services.UserDetailsImpl;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.List;

@RestController
@RequestMapping("/api/users")
public class UserController {

    @Autowired
    private UserRepository userRepository;

    @GetMapping
    public ResponseEntity<List<User>> getAllUsers() {
        List<User> users = userRepository.findAllByUsernameAsc();
        return ResponseEntity.ok(users);
    }
}

```

```

    @GetMapping("/me")
    public ResponseEntity<?> getCurrentUser() {
        Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
        if (authentication != null && authentication.getPrincipal() instanceof UserDetailsImpl) {
            UserDetailsImpl userDetails = (UserDetailsImpl) authentication.getPrincipal();
            return userRepository.findById(userDetails.getId())
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
        }
        return ResponseEntity.status(401).body("Unauthorized");
    }
}

```

ChatMessageController.java

Exposes REST endpoints to query history records, unread message counts, and marks messages as read. It also routes STOMP WebSocket messages.

```

package com.chatapp.controller;

import com.chatapp.dto.MessageDto;
import com.chatapp.model.ChatMessage;
import com.chatapp.model.ChatRoom;
import com.chatapp.model.User;
import com.chatapp.repository.ChatMessageRepository;
import com.chatapp.repository.ChatRoomRepository;
import com.chatapp.repository.UserRepository;
import com.chatapp.security.services.UserDetailsImpl;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.messaging.handler.annotation.Payload;
import org.springframework.messaging.simp.SimpMessagingTemplate;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.bind.annotation.*;
import java.security.Principal;
import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

@RestController
public class ChatMessageController {

    @Autowired
    private ChatMessageRepository chatMessageRepository;

```

```

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private ChatRoomRepository chatRoomRepository;

    @Autowired
    private SimpMessagingTemplate messagingTemplate;

    @MessageMapping("/chat.sendMessage")
    public void sendMessage(@Payload MessageDto messageDto, Principal principal) {
        if (principal == null) {
            return;
        }
        String senderUsername = principal.getName();
        User sender = userRepository.findByUsername(senderUsername)
            .orElseThrow(() -> new RuntimeException("Sender not found"));

        ChatMessage chatMessage = new ChatMessage();
        chatMessage.setContent(messageDto.getContent());
        chatMessage.setSender(sender);

        if (messageDto.getChatRoomId() != null) {
            ChatRoom chatRoom = chatRoomRepository.findById(messageDto.getChatRoomId())
                .orElseThrow(() -> new RuntimeException("Chat room not found"));
            chatMessage.setChatRoom(chatRoom);
            ChatMessage saved = chatMessageRepository.save(chatMessage);

            messageDto.setId(saved.getId());
            messageDto.setTimestamp(saved.getTimestamp());
            messageDto.setSenderAvatar(sender.getAvatarUrl());

            messagingTemplate.convertAndSend("/topic/room." + chatRoom.getId(), messageDto);
        }
        else if (messageDto.getRecipientUsername() != null) {
            User recipient = userRepository.findByUsername(messageDto.getRecipientUsername())
                .orElseThrow(() -> new RuntimeException("Recipient not found"));
            chatMessage.setRecipient(recipient);
            ChatMessage saved = chatMessageRepository.save(chatMessage);

            messageDto.setId(saved.getId());
            messageDto.setTimestamp(saved.getTimestamp());
            messageDto.setSenderAvatar(sender.getAvatarUrl());
            messageDto.setRead(false);

            messagingTemplate.convertAndSendToUser(recipient.getUsername(), "/queue/messages", messageDto);
            messagingTemplate.convertAndSendToUser(sender.getUsername(), "/queue/messages", messageDto);
        }
    }
}

```

```

@GetMapping("/api/messages/room/{roomId}")
public ResponseEntity<List<MessageDto>> getRoomMessages(@PathVariable Long roomId) {
    List<ChatMessage> messages = chatMessageRepository.findByChatRoomIdOrderByTimestampAsc(roomId);
    List<MessageDto> dtos = messages.stream().map(this::convertToDto).collect(Collectors.toList());
    return ResponseEntity.ok(dtos);
}

@GetMapping("/api/messages/direct/{username}")
public ResponseEntity<List<MessageDto>> getDirectMessages(@PathVariable String username) {
    Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
    if (authentication != null && authentication.getPrincipal() instanceof UserDetailsImpl) {
        UserDetailsImpl userDetails = (UserDetailsImpl) authentication.getPrincipal();
        User currentUser = userRepository.findById(userDetails.getId()).orElseThrow();
        User otherUser = userRepository.findByUsername(username)
            .orElseThrow(() -> new RuntimeException("User not found"));

        chatMessageRepository.markAsRead(otherUser.getId(), currentUser.getId());

        List<ChatMessage> messages = chatMessageRepository.findDirectMessages(currentUser.getId(),
            otherUser.getId());
        List<MessageDto> dtos = messages.stream().map(this::convertToDto).collect(Collectors.toList());
        return ResponseEntity.ok(dtos);
    }
    return ResponseEntity.status(401).build();
}

@PostMapping("/api/messages/direct/{username}/read")
public ResponseEntity<?> markMessagesAsRead(@PathVariable String username) {
    Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
    if (authentication != null && authentication.getPrincipal() instanceof UserDetailsImpl) {
        UserDetailsImpl userDetails = (UserDetailsImpl) authentication.getPrincipal();
        User currentUser = userRepository.findById(userDetails.getId()).orElseThrow();
        User otherUser = userRepository.findByUsername(username)
            .orElseThrow(() -> new RuntimeException("User not found"));

        chatMessageRepository.markAsRead(otherUser.getId(), currentUser.getId());
        return ResponseEntity.ok().build();
    }
    return ResponseEntity.status(401).build();
}

@GetMapping("/api/messages/unread")
public ResponseEntity<Map<String, Long>> getUnreadCounts() {
    Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
    if (authentication != null && authentication.getPrincipal() instanceof UserDetailsImpl) {
        UserDetailsImpl userDetails = (UserDetailsImpl) authentication.getPrincipal();
        User currentUser = userRepository.findById(userDetails.getId()).orElseThrow();
        List<User> users = userRepository.findAll();
        Map<String, Long> unreadCounts = new HashMap<>();

        for (User user : users) {
            if (!user.getUsername().equals(currentUser.getUsername())) {

```

```

        long count = chatMessageRepository.countUnreadMessages(user.getId(), currentUser.g
        if (count > 0) {
            unreadCounts.put(user.getUsername(), count);
        }
    }
}
return ResponseEntity.ok(unreadCounts);
}
return ResponseEntity.status(401).build();
}
}

private MessageDto convertToDto(ChatMessage msg) {
    return MessageDto.builder()
        .id(msg.getId())
        .content(msg.getContent())
        .senderUsername(msg.getSender().getUsername())
        .senderAvatar(msg.getSender().getAvatarUrl())
        .recipientUsername(msg.getRecipient() != null ? msg.getRecipient().getUsername() : nul
        .chatRoomId(msg.getChatRoom() != null ? msg.getChatRoom().getId() : null)
        .timestamp(msg.getTimestamp())
        .isRead(msg.isRead())
        .build();
}
}
}

```

2. WebSocket & Presence Configurations

WebSocketConfig.java

Enables STOMP WebSocket broker and registers JWT secure auth handshake interceptors.

```

package com.chatapp.config;

import com.chatapp.security.jwt.JwtUtils;
import com.chatapp.security.services.UserDetailsServiceImpl;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.Ordered;
import org.springframework.core.annotation.Order;
import org.springframework.messaging.Message;
import org.springframework.messaging.MessageChannel;
import org.springframework.messaging.simp.config.ChannelRegistration;
import org.springframework.messaging.simp.config.MessageBrokerRegistry;
import org.springframework.messaging.simp.stomp.StompCommand;
import org.springframework.messaging.simp.stomp.StompHeaderAccessor;
import org.springframework.messaging.support.ChannelInterceptor;
import org.springframework.messaging.support.MessageHeaderAccessor;

```



```

import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.web.socket.config.annotation.EnableWebSocketMessageBroker;
import org.springframework.web.socket.config.annotation.StompEndpointRegistry;
import org.springframework.web.socket.config.annotation.WebSocketMessageBrokerConfigurer;

@Configuration
@EnableWebSocketMessageBroker
@Order(Ordered.HIGHEST_PRECEDENCE + 99)
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Autowired
    private JwtUtils jwtUtils;

    @Autowired
    private UserDetailsServiceImpl userDetailsService;

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws")
            .setAllowedOriginPatterns("*")
            .withSockJS();
        registry.addEndpoint("/ws")
            .setAllowedOriginPatterns("*");
    }

    @Override
    public void configureMessageBroker(MessageBrokerRegistry registry) {
        registry.enableSimpleBroker("/topic", "/queue", "/user");
        registry.setApplicationDestinationPrefixes("/app");
        registry.setUserDestinationPrefix("/user");
    }

    @Override
    public void configureClientInboundChannel(ChannelRegistration registration) {
        registration.interceptors(new ChannelInterceptor() {
            @Override
            public Message<?> preSend(Message<?> message, MessageChannel channel) {
                StompHeaderAccessor accessor = MessageHeaderAccessor.getAccessor(message, StompHeaderAccessor.class);
                if (accessor != null && StompCommand.CONNECT.equals(accessor.getCommand())) {
                    String authHeader = accessor.getFirstNativeHeader("Authorization");
                    if (authHeader != null && authHeader.startsWith("Bearer ")) {
                        String token = authHeader.substring(7);
                        if (jwtUtils.validateJwtToken(token)) {
                            String username = jwtUtils.getUserNameFromJwtToken(token);
                            UserDetails userDetails = userDetailsService.loadUserByUsername(username);
                            UsernamePasswordAuthenticationToken authentication =
                                new UsernamePasswordAuthenticationToken(userDetails, null, userDetails.getAuthorities());
                            accessor.setUser(authentication);
                        }
                    }
                }
                return message;
            }
        });
    }
}

```

```

        }
        return message;
    }
    });
}
}

```

WebSocketEventListener.java

Listens to active connection disconnections and broadcasts presence updates ([ONLINE/OFFLINE](#)) dynamically to all logged-in clients.

```

package com.chatapp.config;

import com.chatapp.dto.MessageDto;
import com.chatapp.model.UserStatus;
import com.chatapp.repository.UserRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.event.EventListener;
import org.springframework.messaging.simp.SimpMessageSendingOperations;
import org.springframework.messaging.simp.stomp.StompHeaderAccessor;
import org.springframework.stereotype.Component;
import org.springframework.web.socket.messaging.SessionConnectedEvent;
import org.springframework.web.socket.messaging.SessionDisconnectEvent;
import java.security.Principal;
import java.time.LocalDateTime;

@Component
public class WebSocketEventListener {

    private static final Logger logger = LoggerFactory.getLogger(WebSocketEventListener.class);

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private SimpMessageSendingOperations messagingTemplate;

    @EventListener
    public void handleWebSocketConnectListener(SessionConnectedEvent event) {
        StompHeaderAccessor headerAccessor = StompHeaderAccessor.wrap(event.getMessage());
        Principal principal = headerAccessor.getUser();
        if (principal != null) {
            String username = principal.getName();
            logger.info("User connected: {}", username);
            userRepository.findByUsername(username).ifPresent(user -> {
                user.setStatus(UserStatus.ONLINE);
            });
        }
    }
}

```

```

        userRepository.save(user);
    }

    MessageDto presenceMessage = MessageDto.builder()
        .content("ONLINE")
        .senderUsername(username)
        .timestamp(LocalDateTime.now())
        .build();
    messagingTemplate.convertAndSend("/topic/presence", presenceMessage);
    });
    }
}

@EventListener
public void handleWebSocketDisconnectListener(SessionDisconnectEvent event) {
    StompHeaderAccessor headerAccessor = StompHeaderAccessor.wrap(event.getMessage());
    Principal principal = headerAccessor.getUser();
    if (principal != null) {
        String username = principal.getName();
        logger.info("User disconnected: {}", username);

        userRepository.findByUsername(username).ifPresent(user -> {
            user.setStatus(UserStatus.OFFLINE);
            userRepository.save(user);
        });

        MessageDto presenceMessage = MessageDto.builder()
            .content("OFFLINE")
            .senderUsername(username)
            .timestamp(LocalDateTime.now())
            .build();
        messagingTemplate.convertAndSend("/topic/presence", presenceMessage);
    });
    }
}
}

```

3. Endpoint Routing Maps & Channel Broker Destinations

REST APIs:

- **POST /api/auth/register**: Registers new users with a custom avatar gradient.
- **POST /api/auth/login**: Validates credentials and returns JWT bearer token.
- **POST /api/auth/logout**: Updates user status to **OFFLINE** and clears session context.
- **GET /api/rooms**: Returns catalog of public rooms.
- **POST /api/rooms**: Creates a new public chat room.
- **GET /api/users**: Returns complete list of users and active presences.

- **GET /api/messages/room/{roomId}**: Retrieves historical message list for a specific chat room.
- **GET /api/messages/direct/{username}**: Retrieves message log for a private direct message conversation and resets unread flags.
- **POST /api/messages/direct/{username}/read**: Resets unread status counts manually.
- **GET /api/messages/unread**: Returns key-value pairs of unread counts from other active senders.

WebSocket Topics:

- **CONNECT Headers**: Requires **Authorization: Bearer <token>** STOMP connection frame parameter.
- **Destination /app/chat.sendMessage**: Submits messages (JSON payload containing room ID or recipient username).
- **Subscribe /topic/room.{roomId}**: Receives live broadcast updates for a public room.
- **Subscribe /user/queue/messages**: Receives secure, private direct messages sent from peers.
- **Subscribe /topic/presence**: Captures realtime user status presence broadcasts (**ONLINE/OFFLINE**).